

Name : Narendra Kumar S

Date : 21-08-2025

Task :

Build a Web API project from scratch with the following requirements:

1. Create a table to store the values
2. Insert data into the table should only be done by the Admin
3. View the data from the table by the Officer

Solution:

Program.cs:

```
using Day13.Context;
```

```
using Day13.Security;
```

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
```

```
using Microsoft.EntityFrameworkCore;
```

```
using Microsoft.IdentityModel.Tokens;
```

```
using Microsoft.OpenApi.Models;
```

```
using System.Text;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection("jwt"));
```

```
var jwt = builder.Configuration.GetSection("jwt").Get<JwtOptions>(!);
```

```
var connectionString = builder.Configuration["sql_con:connection"];
```

```
builder.Services.AddDbContext<AppDbContext>(options =>
```

```
options.UseSqlServer(connectionString));
```

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.RequireHttpsMetadata = false;
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateIssuerSigningKey = true,
            ValidateLifetime = true,
            ValidIssuer = jwt.Issuer,
            ValidAudience = jwt.Audience,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.Key)),
            ClockSkew = TimeSpan.Zero
        };
    });
```

```
builder.Services.AddAuthorization();
```

```
builder.Services.AddControllers();
```

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen(c =>
```

```
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "Day13 API", Version = "v1" });
```

```

c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
{
    Name = "Authorization",
    Type = SecuritySchemeType.ApiKey,
    Scheme = "Bearer",
    BearerFormat = "JWT",
    In = ParameterLocation.Header,

    Description = "Enter 'Bearer' [space] and then your valid JWT token.\r\n\r\nExample:
\"Bearer eyJhbGciOiJIUzI1NiIs...\"";

});

```

```

c.AddSecurityRequirement(new OpenApiSecurityRequirement
{
    {
        new OpenApiSecurityScheme
        {
            Reference = new OpenApiReference
            {
                Type = ReferenceType.SecurityScheme,
                Id = "Bearer"
            }
        },
        new string[] {}
    }
});

});

```

```
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Model :

```
namespace Day13.Model
{
    public class StateInfo
    {
        public int Id { get; set; }
        public string State { get; set; }
        public string ChiefMinister { get; set; }
        public string Language { get; set; }
    }
}
```

```

namespace Day13.Model
{
    public class Users
    {
        public int Id { get; set; }

        public string Name { get; set; }

        public string Email { get; set; }

        public string Password { get; set; }

        public string Role { get; set; } = "Officer";
    }
}

```

Controller:

```

using Day13.Context;

using Day13.Model;

using Microsoft.AspNetCore.Authorization;

using Microsoft.AspNetCore.Mvc;

```

```

namespace Day13.Controllers
{
    [ApiController]

    [Route("[controller]")]

    public class StateController : ControllerBase
    {

```

```
private readonly AppDBContext _db;

public StateController(AppDBContext db)
{
    _db = db;
}

[HttpPost("AddState")]
[Authorize(Roles = "Admin")]
public IActionResult AddState(StateInfo stateInfo)
{
    _db.StateInfos.Add(stateInfo);
    _db.SaveChanges();
    return Ok("State Added Successfully");
}

[HttpGet("GetStates")]
[Authorize(Roles = "Officer")]
public IActionResult GetStates()
{
    var states = _db.StateInfos.ToList();
    return Ok(states);
}
}
```

```
}
```

User Controller

```
using Day13.Context;
```

```
using Day13.Model;
```

```
using Day13.Security;
```

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.Extensions.Options;
```

```
using Microsoft.IdentityModel.Tokens;
```

```
using System.IdentityModel.Tokens.Jwt;
```

```
using System.Security.Claims;
```

```
using System.Security.Cryptography;
```

```
using System.Text;
```

```
namespace Day13.Controllers
```

```
{
```

```
    [ApiController]
```

```
    [Route("[controller]")]
```

```
    public class UserController : ControllerBase
```

```
    {
```

```
        private readonly AppDbContext _db;
```

```
        private readonly JwtOptions _jwt;
```

```
        public UserController(AppDbContext db, IOptions<JwtOptions> jwtOptions)
```

```
{  
    _db = db;  
    _jwt = jwtOptions.Value;  
}
```

```
[HttpPost("Register")]
```

```
public IActionResult Register(Users user)
```

```
{  
    user.Password = Hash(user.Password);  
    _db.Users.Add(user);  
    _db.SaveChanges();  
    return Ok("User registered");  
}
```

```
[HttpPost("Login")]
```

```
public IActionResult Login(LoginRequest login)
```

```
{  
    var hashed = Hash(login.Password);  
    var user = _db.Users.FirstOrDefault(u => u.Name == login.Name && u.Password ==  
hashed);  
    if (user == null)  
        return Unauthorized("Sorry Not Authenticated");  
  
    var claims = new List<Claim>  
    {
```



```
        new Claim(ClaimTypes.Name, user.Name),  
        new Claim(ClaimTypes.Role, user.Role ?? "Officer")  
    };
```

```
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwt.Key));  
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
```

```
var token = new JwtSecurityToken(  
    issuer: _jwt.Issuer,  
    audience: _jwt.Audience,  
    claims: claims,  
    expires: DateTime.UtcNow.AddMinutes(_jwt.ExpireInMinutes),  
    signingCredentials: creds  
);
```

```
var jwt = new JwtSecurityTokenHandler().WriteToken(token);  
return Ok(new { token = jwt, expires = token.ValidTo });  
}
```

```
public record LoginRequest(string Name, string Password);
```

```
private static string Hash(string raw)  
{  
    using var sha = SHA256.Create();
```

```

        var bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(raw));

        var sb = new StringBuilder(bytes.Length * 2);

        foreach (var b in bytes) sb.Append(b.ToString("x2"));

        return sb.ToString();
    }
}
}

```

Appsetting

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "jwt": {
    "Issuer": "www.payoda.com",
    "Audience": "students",
    "Key":
"DQWDzxcsedHGCCtyvJVHJHvjsjgAGJSasfsdfsdfsw-0eif=3-qfjeiwqjf39fh39qwpifqpo3wmnch
-9p2rhoiefoiwherfbouewjsjhSS",
    "ExpireInMinutes": 60
  },
  "sql_con": {

```

```
"connection": "Data Source=PTPLL614;Initial Catalog=TrainingDB;Integrated Security=true;TrustServerCertificate=True"
```

```
},
```

```
"AllowedHosts": ""
```

```
}
```

AppDbContext

```
using Day13.Model;
```

```
using Microsoft.EntityFrameworkCore;
```

```
namespace Day13.Context
```

```
{
```

```
    public class AppDbContext : DbContext
```

```
    {
```

```
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
```

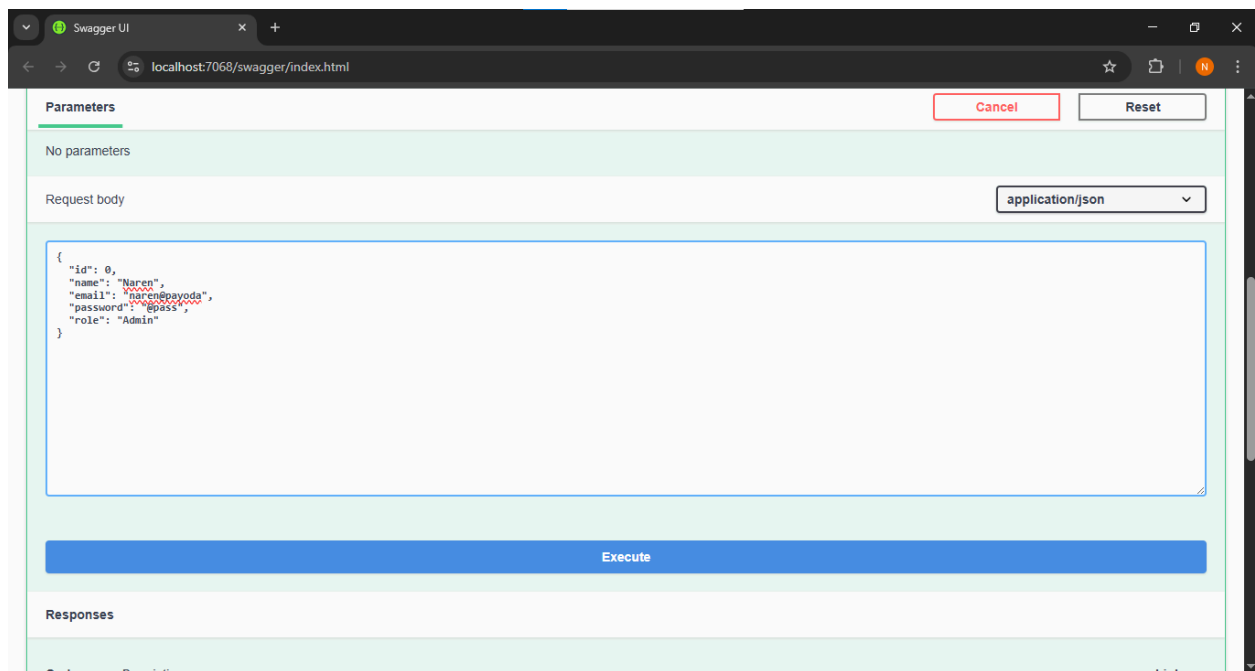
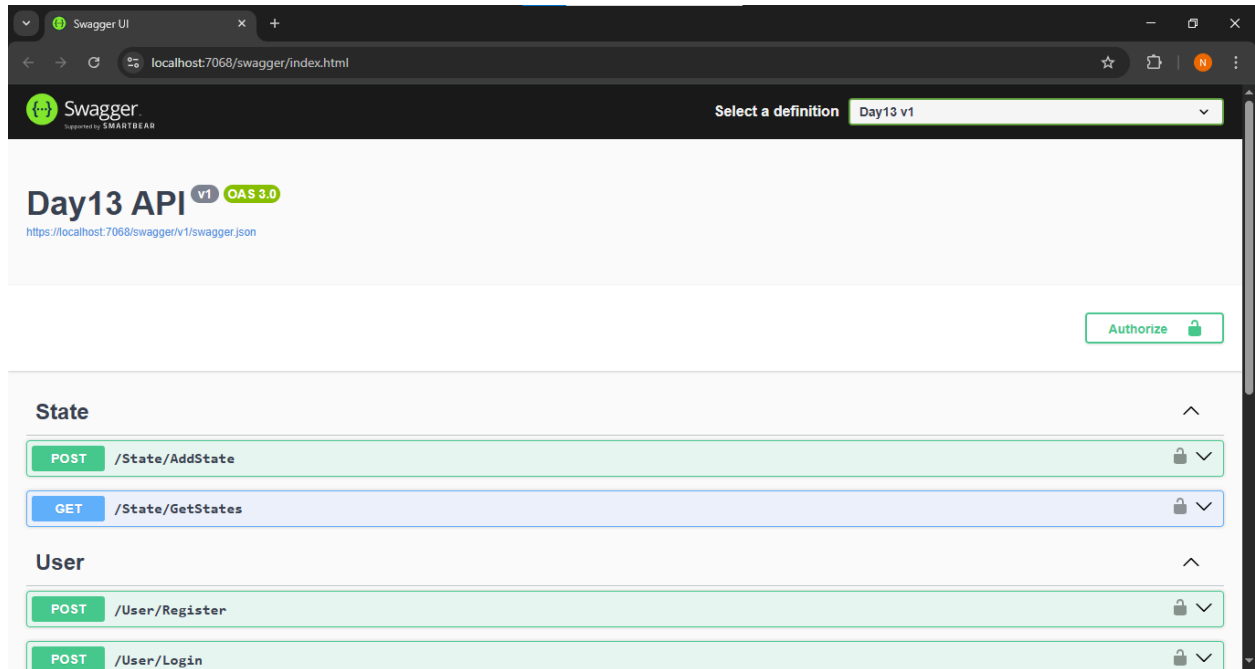
```
        public DbSet<Users> Users { get; set; }
```

```
        public DbSet<StateInfo> StateInfos { get; set; }
```

```
    }
```

```
}
```

Output :



Swagger UI

localhost:7068/swagger/index.html

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7068/User/Register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 0,
    "name": "naren",
    "email": "naren@payoda",
    "password": "@pass",
    "role": "Admin"
  }'
```

Request URL

https://localhost:7068/User/Register

Server response

Code	Details
200	Response body User registered Response headers content-type: text/plain; charset=utf-8 date: Mon, 25 Aug 2025 13:44:51 GMT server: Kestrel

Responses

Code	Description	Links
------	-------------	-------

PTPL614.TrainingDB - dbo.Users - Microsoft SQL Server Management Studio

Quick Launch (Ctrl+Q)

File Edit View Project Query Designer Tools Window Help

master Execute

Object Explorer

- Database Diagrams
- Tables
 - System Tables
 - File Tables
 - External Tables
 - Graph Tables
 - dbo.__EFMigrationsHistory
 - dbo.agri
 - dbo.Authors
 - dbo.Books
 - dbo.Movies
 - dbo.NewBooks
 - dbo.Product
 - dbo.Sales
 - dbo.StatInfos
 - dbo.Student
 - dbo.tree
 - dbo.Users
 - Columns
 - Keys
 - Constraints
 - Triggers
 - Indexes
 - Statistics
 - Dropped Ledger Tables

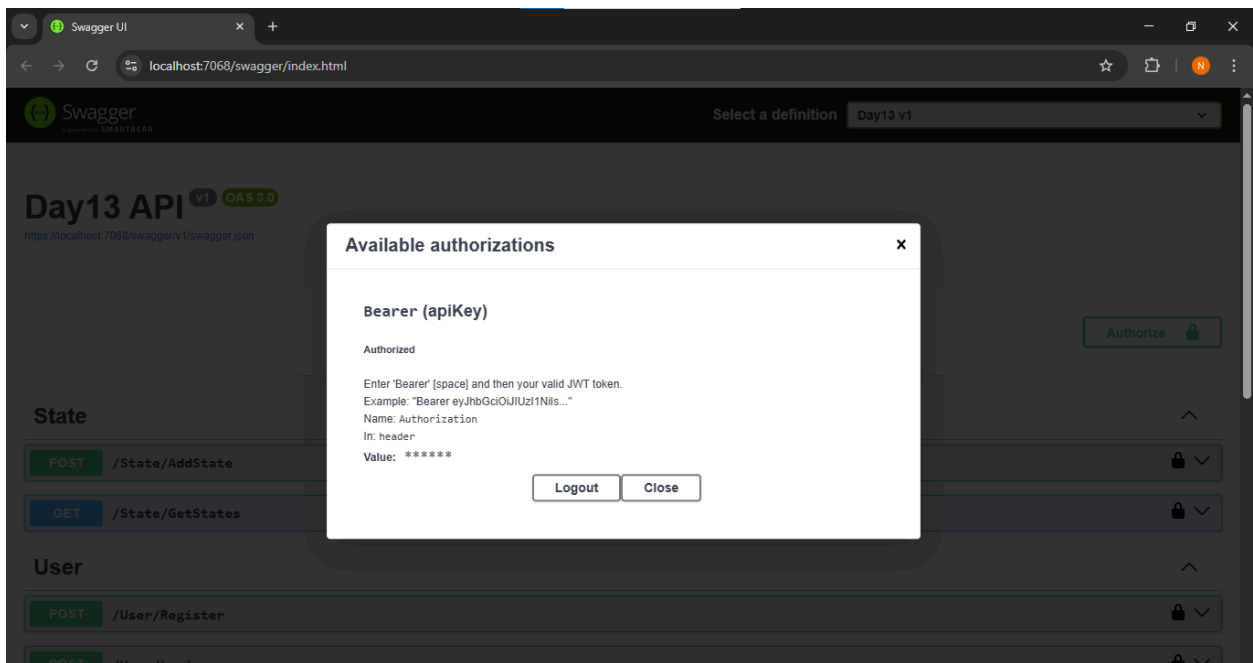
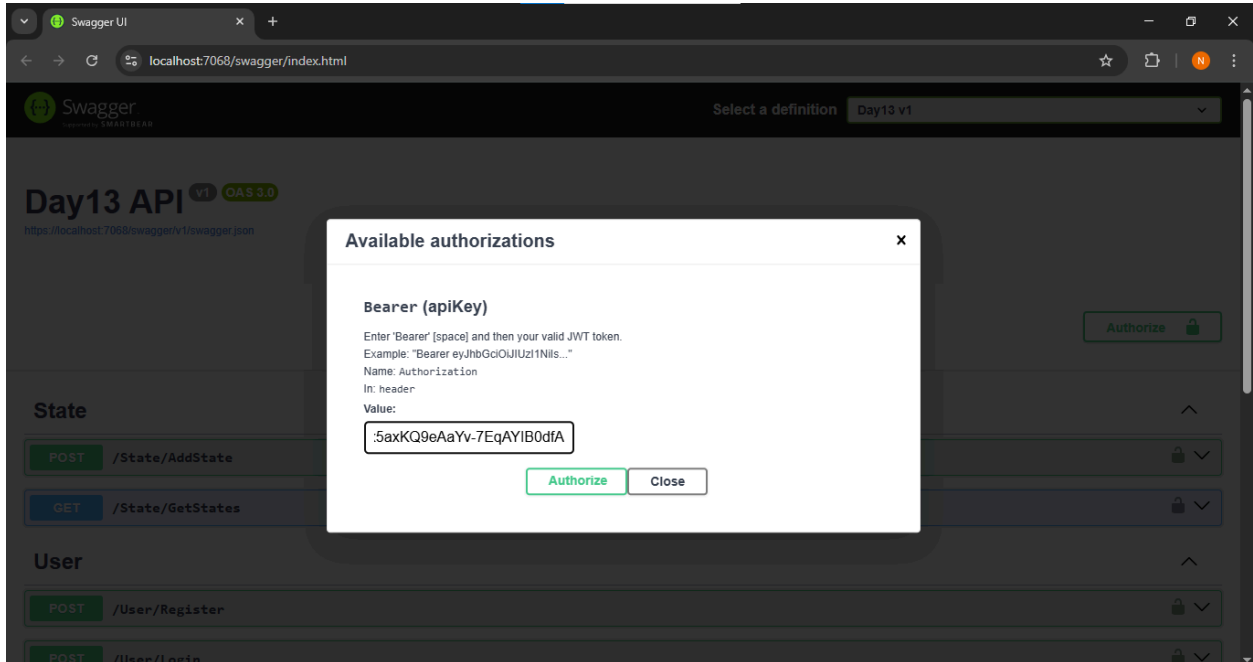
PTPL614.TrainingDB - dbo.Users

ID	Name	Email	Password	Role
1	Naren	naren@payoda	@pass	Admin
2	NULL	NULL	NULL	NULL

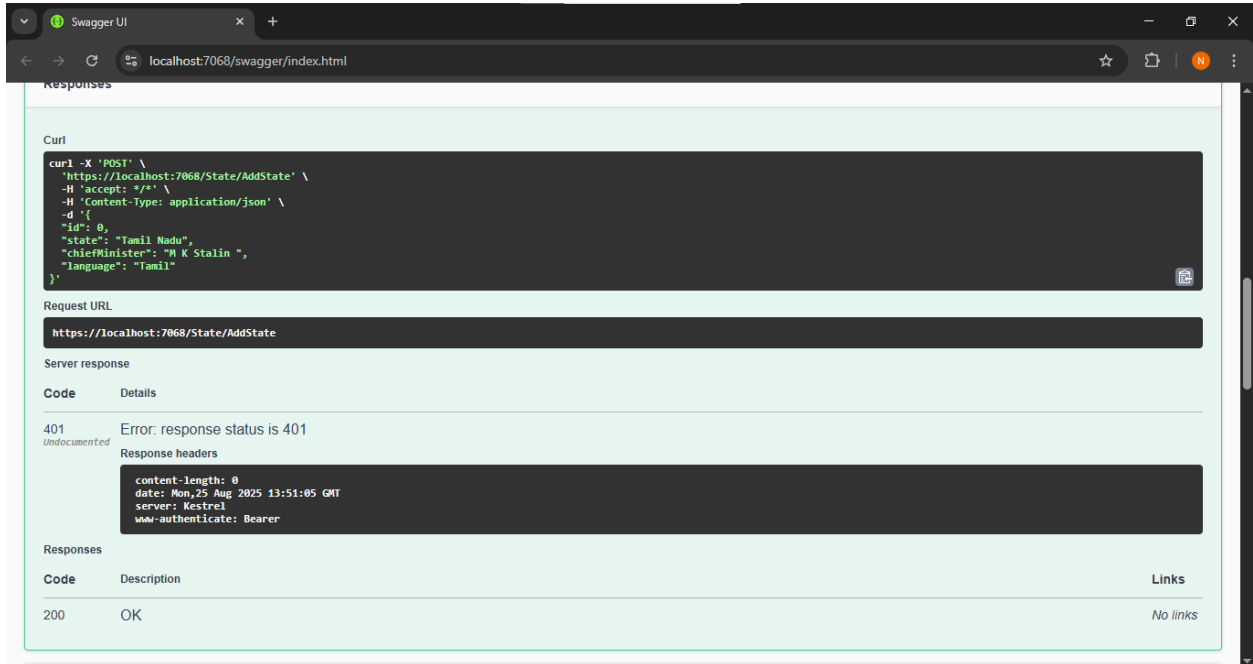
SQLQuery1.sql - PT...rendrakumar.s (56) *

Ready

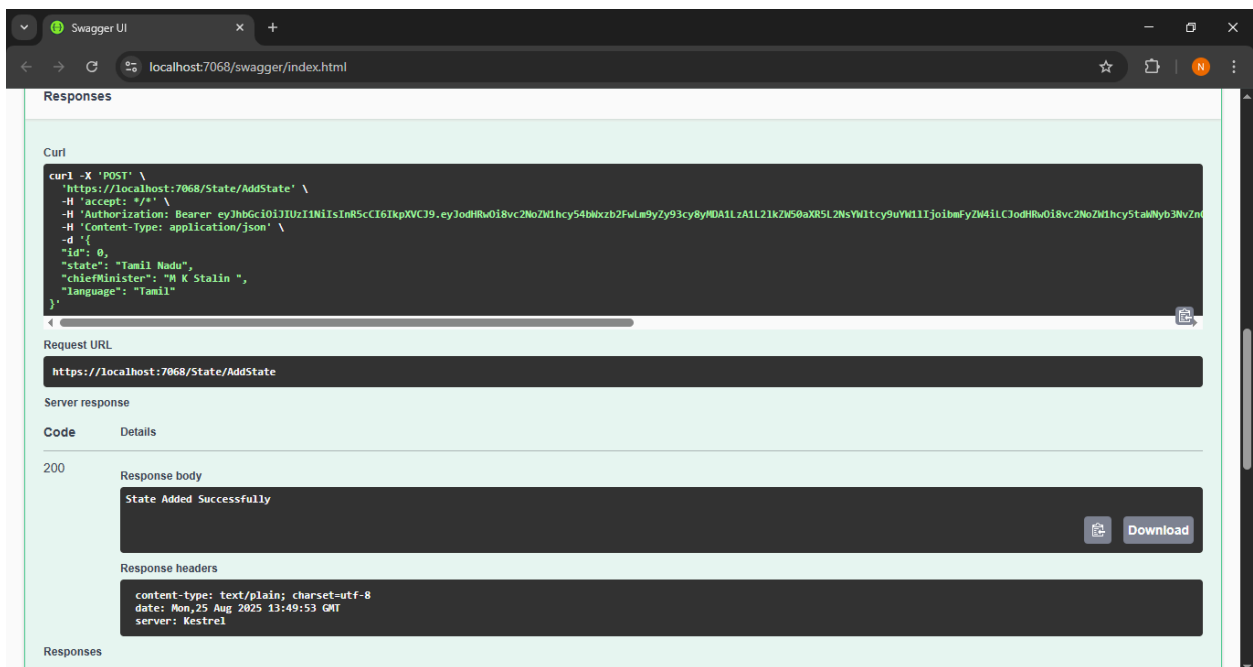
Validate Page:



Unauthorized Insert error message :



Valid User Insert Details Output:



Officer Login Output:

POST /User/Register

Parameters

No parameters

Request body

application/json

```
{
  "id": 0,
  "name": "Kumar",
  "email": "kumar@test.com",
  "password": "@pass",
  "role": "Officer"
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7068/User/Register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 0,
    "name": "Kumar",
    "email": "kumar@test.com",
    "password": "@pass",
    "role": "Officer"
  }'
```

Request URL

https://localhost:7068/User/Register

Server response

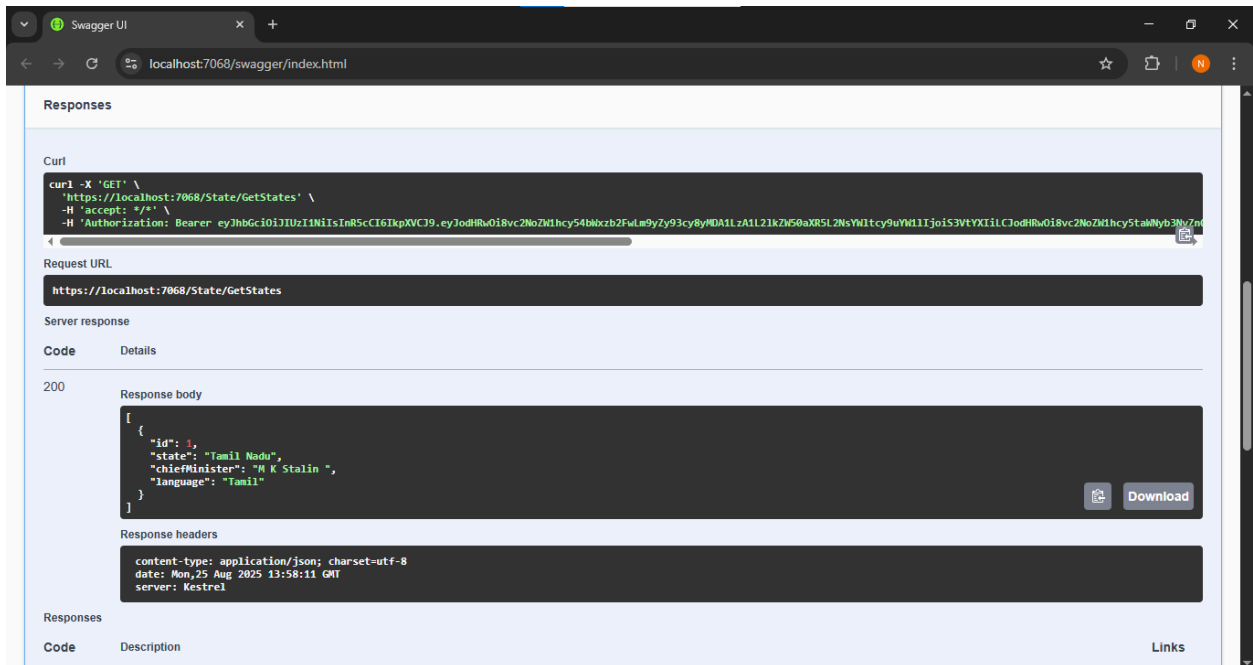
Code	Details
200	Response body User registered Response headers content-type: text/plain; charset=utf-8 date: Mon, 25 Aug 2025 13:54:25 GMT server: Kestrel

Responses

Code	Description	Links
------	-------------	-------

Login with Officer credentials:

The screenshot shows a web browser window with the Swagger UI for a REST API. The browser address bar displays 'localhost:7068/swagger/index.html'. The Swagger UI interface includes a 'Curl' section with a cURL command for a POST request to 'https://localhost:7068/User/Login'. Below this, the 'Request URL' is 'https://localhost:7068/User/Login'. The 'Server response' section shows a 200 status code. The 'Response body' is a JSON object containing a 'token' and an 'expires' timestamp. The 'Response headers' section shows 'content-type: application/json; charset=utf-8', 'date: Mon, 25 Aug 2025 13:56:43 GMT', and 'server: Kestrel'. The 'Responses' table at the bottom shows a 200 status code with the description 'OK'.



Output Screenshot for Invalid credentials:

